

Super Restore: A Time Series Analysis of Runescape's Most Popular Potion

Kaylee Vo

kav418@g.harvard.edu

ALM Data Science, Harvard University

Abstract

Runescape is a widely popular massively multiplayer online game (MMO) peaking in popularity in the early 2000's. For its level of popularity, little empirical research has been conducted on the economics of the game's Grand Exchange (GE). In this work, we analyze the economic dynamics of Super Restore, one of the game's most popular potions. We apply time series models to determine whether the GE behaves similarly to real-world exchanges. We construct a DAG for Super Restore's value chain and use regression models to estimate the edges from input good to output good. In doing so, we also show that there is no evidence of a wealth effect in Runescape's virtual prices. We then use a vector error correction model (VECM), combined with structural breaks, to show evidence of a substitution relationship between Prayer Potions and Super Restore. We further support this claim by estimating the impulse response functions using vector autoregression (VAR), including all items in the value chain as endogenous variables. We then explore a wide gamut of models to determine the model that best forecasts Super Restore prices. We show that a long short-term memory neural network (LSTM) with two hidden layers outperforms all other models in predicting Super Restore prices. Finally, we conclude by recommending ARIMA as the optimal model due to its simplicity and close performance to LSTM.

1 Introduction

1.1 Game Context

Runescape is a massively multiplayer online game (MMO) set in the European medieval era. The game peaked in popularity around 2007 and slowly declined in popularity from then on (Figure 1). However, the game is still very popular today with an average of 80,000 daily active users (Server Population & Player Count - MMO Populations, 2001).

In the game, players interact, embark on quests, level up skills, and most importantly for this project, trade items through a platform known as The Grand Exchange (GE). The GE is analogous to the real-world exchange. Players can sell almost any item at any price (p_s). For a seller, if the exchange is able to find a buyer with a buying price p_b , and if $p_b \geq p_s$, a trade occurs at p_s . Any difference is returned to the buyer. In the case of multiple competing offers, the GE operates on a first-in, first-out basis. In other words, older offers are satisfied first.

The GE induces market dynamics within the game. An example of such dynamics is if the price for a good goes up, players may enter the market, driving prices down. Using a concrete example, if the price of coal goes up, more players will take up mining, increasing supply and thus driving prices down. Additionally, in Runescape some items are used to create other goods. As such, the game has a notion of raw, intermediate, and final goods. This relationship allows us to analyze how a shock in price will affect the price of downstream goods.

The last piece of important context is the game's currency. Runescape's virtual currency is called 'gold pieces' or 'gp' for short. Gp can be earned in-game through a myriad of options, such as completing quests or killing monsters. As with most games, players can use real-world currency to purchase in-game currency. Uniquely, the reverse is also true. Players can use gp to buy membership, which has a real-world price. The existence of in-game purchases allows us to measure the possibility of a wealth effect in Super Restore prices.

1.2 Motivation

Very little empirical research has been done on Runescape’s GE data, making it ripe for analysis. On Kaggle, there are only a few notebooks and none of them focus on forecasting. A quick Google scholar search also shows that Runescape data has not been investigated in any empirical way. The hope for this project is to draw inferences with Vector Autoregressive Models (VAR) and VECM models and to determine a model that accurately forecasts Super Restore prices. Our choice for using Super Restore over all other items was motivated by 1) the item’s popularity and 2) an exploratory data analysis showed a well-behaved series with few random spikes and abrupt changes.

This project will proceed as follows. We present the research questions, discuss the data, and then proceed with modeling. In the modeling section, we show evidence of a substitution relationship between Super Restore and Prayer Potion. We also show the non-existence of a wealth effect and estimate the edges in the value chain DAG. We conclude with a discussion of the results and future work. All tables and figures referred are located in Appendix A. Appendix B contains math notation, which supplements the concepts introduced in the main sections.

2 Research Questions

The data is vast; therefore we restrict the research questions to make the project tractable. The research questions and initial hypotheses are as follows:

1. Which model best forecasts Super Restore prices?
2. Are Prayer Potions and Super Restore Potions complement or substitute goods?
3. Is there evidence of a wealth effect on the price of Super Restore?
4. Do items in the virtual Runescape universe follow economic intuition? In other words, are the edges in the value chain positive and significant?

Our initial hypotheses are the following:

1. The best model for forecasting Super Restore prices will be ARIMA since this model has done consistently well across the homework assignments.
2. Prayer Potions and Super Restore Potions are substitutes because they heal similar player stats. On Reddit forums, players would often suggest buying Prayer Potions over Super Restore because the extra benefits that Super Restore offers are unnecessary.
3. There is no wealth effect on the price of Super Restore because few people purchase in-game gold.
4. The items in the value chain will follow economic intuition. The price of input goods and final goods are positively and significant.

3 Data

3.1 Overview

The data for this project was downloaded from Kaggle and contains daily frequency price data, ranging from "2008-05-20" to "2024-09-19" (Runescape Grand Exchange Data, 2024). The raw data is long with 26,246,910 observations and five features. In total, there are 6185 unique items. The schema of the raw data is shown in Table 1. Given the size of the data, the project will focus on five items: Prayer Potions, Super Restore Potions, Snapdragon Seed, Snapdragon, and Red Spiders’ Eggs. Additionally, we augment the Runescape data with the daily S&P 500 index and the daily 10-year treasury yield from FRED to measure the possible existence of a wealth effect (Federal Reserve Economic Data || FRED || St. Louis Fed, n.d.). These two features were chosen because the frequency is daily and each have been shown to be adequate proxies for measuring macroeconomic health (Parnes, 2019; King & Watson, 1996). The schema of the training set and test set is shown in Table 2.

Regarding data usability, the data is clean with no missing values for the “price” feature. All duplicate daily records were averaged. Note, not all days had data but this does not pose a challenge to our analysis. The only feature with a significant amount of missing data was “volume”, with around 80% of it missing (Figure 2). Therefore, we removed this feature. Furthermore, we trim the date range to be from “2015-01-01” to “2022-12-31” due to noisy data in recent and older years. Trimming the data in this fashion also aligns Runescape’s data with FRED’s. After data processing, the final data set has a total of 14,520 observations. For the test set, we hold out the last two weeks of 2022, following a similar convention to prior homework assignments.

3.2 Exploratory Data Analysis

As with any rigorous data science research, we conducted a brief exploratory data analysis (EDA) before executing any empirical method. There are a few important takeaways. The first is the positive trend of aggregate prices, which suggests inflation and perhaps the need to account for trends in our series. The second is the growing number of items per year, which is less important with respect to this project given the restricted scope of items. A line plot for each time series is displayed from Figure 6 to Figure 12. Most of the series look reasonable except for the two unusual spikes in Snapdragon Seed and a massive boom and bust in Red Spiders’ Eggs.

For each feature, we conducted an augmented Dickey–Fuller (ADF) test and KPSS test for stationarity (Definition 1). The null and alternative hypothesis for each test are as follows.

ADF Test

- H_0 : There exists a unit root in the time series (the series is not stationary).
- H_1 : There is no unit root in the time series (the series is stationary).

KPSS Test

- H_0 : There is no unit root in the time series (the series is stationary).
- H_1 : There exists a unit root in the time series (the series is not stationary).

In each test, all features except for Snapdragon and S&P 500 used the “constant” parameter. This parameter tells the test to assume the series is stationary around an arbitrary, non-zero mean. Snapdragon and S&P 500 were tested using the “trend” parameter, which means the test checks for non-stationarity while accounting for a deterministic trend. The results of the two tests are shown in Table 3. Both tests agree for all series except for Super Restore. In this case, the ADF test suggests the series is stationary while the KPSS test suggests it is non-stationary. However more credence should be lent to the KPSS test because ADF tests have been shown to have low power (i.e. low probability of avoiding a false negative) in some empirical studies (DeJong et al., 1992). Knowing that most of the series are non-stationary, we’ll need to difference the series during modeling if stationarity is a model assumption.

4 Modeling

Upon completion of the EDA, we now proceed with modeling to answer the research questions outlined. We start with simple models and progressively build towards more complex models. With each model, we iteratively construct evidence to support our conclusions. The first model is a fixed effects model.

4.1 Fixed Effects Model

For our baseline, we use a fixed effects model with two-week and four-week lags. We include fixed effects for year and month. The dependent variable is Super Restore. The effects of interest are β_3 , β_4 , and β_5 , the coefficients on Snapdragon. The covariates P and I are S&P 500 index and the 10-year treasury yield respectively. The entire model specification is shown below.

$$S_t = \beta_1 S_{t-14} + \beta_2 S_{t-28} + \beta_3 D_t + \beta_4 D_{t-14} + \beta_5 D_{t-28} + \beta_6 P_t + \beta_7 I_t + \alpha_y + \gamma_m + \epsilon_t \quad (1)$$

- S : Super Restore price
- D : Snapdragon price
- P : S&P 500 index
- I : 10-year treasury yield
- α_y : Year fixed effects
- γ_m : Month fixed effects

The results of the regression show significance in all covariates except for 10-year treasury yield. The coefficient on the contemporaneous Snapdragon regressor is positive, suggesting an increase in the price of Snapdragon is associated with an intra-day increase in the price of Super Restore and vice-versa. So far, the result shows that Runescape items behave like real-world goods. Examining the coefficients in the lagged values of Snapdragon, seeing that they are negative, suggests a mean-reverting behavior in Snapdragon prices. As for the lags of Super Restore, the coefficients are all positive. This provides evidence of inertia or momentum for Super Restore prices i.e. prior prices influence current prices.

Examining the coefficients for the FRED features, the coefficient on the S&P 500 index is significant and negative. This may suggest evidence of a wealth effect. The direction of the coefficient suggests that when the real-world economy is doing poorly, people have more leisure time for video games. As for the 10-year treasury yield, there was no evidence of wealth effect. One possible explanation is that the effect of interest rate is too indirect to affect decisions related to in-game purchases. Perhaps a better measure would have been shorter term yields, which captures credit card interest rates, since credit cards are a popular payment method for in-game purchases (Gong et al., 2023).

The inclusion of fixed effects accounts for unobservable confounding by assuming confounders are time-invariant (Definition 2). For example, if Super Restore price is influenced by players' income, we can control for it by assuming players' income are constant over time. Additionally, a year fixed effect is assumed to have the same effect across all times, meaning all series receive the same offset.

The adjusted R^2 of the model is 0.988 which is incredibly high, possibly inflated due to the presence of serial correlation. This is supported by the small p-value on the Breusch-Godfrey test and by the slow decay in the lags of the ACF plot (Figure 13). The lags were included in the model to address serial correlation but it is possible that the lag structure chosen isn't correct. Overall, the residuals look symmetric, normal and homoskedastic but not independent.

4.1.1 Test Set Forecast

The statistical inferences in the prior section look promising, so now we forecast Super Restore prices using Model (1). To prevent data leakage, we used the `auto.arima` function to find the best ARIMA model and use it to forecast the predictor values. We do this for Snapdragon prices, the S&P 500 index, and 10-year Treasury yield. We also use the ARIMA model to predict Super Restore prices, since lagged dependent variables were included as regressors.

For the two-week holdout set, the RMSE of Model (1) was 223.882, which is a decent test score for a baseline model. This test score will serve as our baseline for more sophisticated models. A comparison of all model performances can be found in the Results section.

4.1.2 Estimating Edges in the Value Chain

We will now use the lagged fixed effects model to estimate the rest of the edges in the value chain DAG (Figure 5). This estimation tests our research question, confirming whether or not the items in the virtual Runescape universe reflect real-world economic behaviors. Our initial hypothesis is that they do. To confirm our hypothesis, the edge estimates should be

positive and significant because as the cost of an input good increases, the price of the final good should increase as well. Waning demand may push the prices back down, but we should see a positive coefficient for contemporaneous regressors or in recent lags.

The edge from Snapdragon to Super Restore was already estimated by Model (1). The contemporaneous effect was 0.460 and statistically significant. We estimate two more regression models, using the same model specifications, swapping out the dependent variable for a given output good and the independent variable for a given input good. Using R formula notation, we estimate the following models:

- DAG Model 1
 - Super Restore $\sim$ Red Spiders' Eggs
- DAG Model 2
 - Snapdragon $\sim$ Snapdragon Seed.

In DAG Model 1 (Table 6), we see a positive relationship between Red Spiders' Eggs and Super Restore. All subsequent lags are negative and insignificant. In DAG model 2 (Table 7), the coefficient on Snapdragon seed is positive and significant. The two-week lag is negative, while the four-week lag is positive. Both lags are statistically significant. The results are intuitive because a price increase in the input good will lead to a price increase in the output good. However as prices increase, demand for the final good wanes as reflected in the lags, so prices fall back down. In summary, the results show evidence of standard economic behavior, confirming our initial hypothesis for research question (4).

In terms of a wealth effect, all regression models show a significance in at least one FRED feature; however, the signs of the FRED coefficients do not agree across models, therefore we cannot conclude the existence of a wealth effect (Tables 5 to 7). This rejects our initial hypothesis but answers research question (3).

4.2 VAR Model

We shift our focus to research question (2): are Prayer Potions and Super Restore Potions substitute or complement goods? Two goods, A and B are substitutes if a shock in price A leads to a price increase in B because people move away from good A and purchase more of good B . For complement goods, the opposite is true. Similar to other empirical studies, we leverage the impulse response functions given by VAR to test this hypothesis (Gagliardone & Gertler, 2023).

We include five time series into our VAR model: Prayer Potions, Super Restore, Snapdragon, Snapdragon Seed and Red Spiders' Eggs. All series are converted to growth rates because a separate experiment (not shown) using level data led to non-intuitive results. Furthermore, in the EDA section all tests almost unanimously agreed that all series are non-stationary, so growth rate is likely a good choice. In a VAR model, all variables are treated as endogenous. Since the output is a vector, we estimate a set of five regression models, where each feature is expressed as a linear combination of all other variables. Using the estimated VAR's, we generate the following five impulse response functions:

1. Shock of Super Restore on Prayer Potion.
2. Shock of Snapdragon on Super Restore.
3. Shock of Snapdragon Seed on Snapdragon.
4. Shock of Snapdragon Seed on Super Restore
5. Shock of Red Spiders' Eggs on Super Restore.

To select the lags for our VAR model we use the `VARselect()` function. The best lag chosen by AIC is four. For BIC, the best lag is two. BIC is a harsher metric than AIC because it has a larger scaling factor in the model complexity term (Definition 5). For our analysis, we'll use four lags. Our VAR specification is then,

$$Y_t = \nu + A_1 Y_{t-1} + Y_{t-2} + \dots + A_p Y_{t-p} + \epsilon_t. \quad (2)$$

For our case p is 4, Y_t is the vector of endogenous variables, A is a 5×5 matrix of coefficients and ν is a vector of intercept terms (Sims, 1980).

All five impulse response functions are shown in Figures 14 to 18. The impulse response of Prayer Potion to a shock on Super Restore shows a positive increase and reversion to the mean. This suggests the two goods are indeed substitutes because if the price of one good goes up, players shift away from the good to purchase the other. As for the other impulse responses, the results solidify our hypothesis that the value chain reflects real-world economic dynamics. Across the board, we observe that a shock to the price of an input good on the output good initially causes a price increase, then a drop due to decreased demand, and finally a reversion to the mean. One oddity is the impulse response of Super Restore from a Snapdragon Seed price shock. We observe only a decrease and a reversion to the mean. Most likely, the raw good is too far down the value chain to have a strong effect on Super Restore, as indicated by the large confidence interval.

4.2.1 Forecasts

In addition to impulse response functions, VAR models are capable of generating forecasts. We'll use the prior fitted models to predict Super Restore prices. As mentioned, we keep a running tally to compare all models in the Results section. The final model comparison will answer research question (1). The RMSE of the VAR model for Super Restore was 11,079.969, which is incredibly poor. For the other series, the VAR didn't perform as poorly (see notebook). Although the VAR demonstrates a strong ability for inference, it does not perform well as a predictor Super Restore prices.

To conclude this section, we were able to answer and confirm research question (4). The empirical evidence suggests that virtual goods behave like real-world goods. We were also able to provide evidence for research question (2), that Super Restore and Prayer Potions are substitutes.

4.3 VECM Model

To further strengthen the argument that Super Restore and Prayer Potions are substitutes we employ a VECM model. The general idea of a VECM is that two series are non-stationary but there exists a linear combination of the two series that is stationary, e.g. a series with integration order 0. We now conduct the following two experiments:

Experiment 1

- We truncate the two series at "2020-09-28". From Figure 19, we see the two series diverge. This breakpoint will be substantiated in the next section. Our initial hypothesis is that the error correction term in the model is significant.

Experiment 2

- We run the same procedure as before, but instead use the complete series. Our initial hypothesis is that the significance of the error correction term vanishes.

The model specification for VECM is then

$$\Delta y_t = \Pi y_{t-1} + \sum_{i=1}^{p-1} \Gamma_i \Delta y_{t-i} + u_t.$$

More details can be found in B.2: A plot of the two series and the difference between them is shown in Figure 19.

4.3.1 Experiment 1 Results

To select the number of lags for the VECM we use the `VAR()` function, with AIC as the criteria. The optimal number of lags chosen by this function is 8. To test the existence of cointegration between the two series, we apply the Johansen-Procedure. We use the Elliott-Rothenberg-Stock (ERS) test to check if the linear combination of the two series is stationary (Mohr, n.d.).

The Johansen-Procedure concludes there is no cointegration while the ERS test concludes the error correction term is stationary. To estimate the VECM we use `VECM()` using lags set to 8 and a cointegration level (r) of 1. The regression outputs are shown in Table 8. The regression results show a significant error correction term for the Super Restore equation but not the Prayer Potion equation. This implies that prayer potion plays a vital role in correcting the Super Restore series but not the other way around.

4.3.2 Experiment 2 Results

Now we run the same procedure but instead we use the entire series, with the time period after the structural break. The Johansen-Procedure concludes that the two series are not cointegrated, which is expected. The ERS test concludes that the linear combination of the two series is non-stationary.

In the estimation of the VECM, the significance of the error correction term vanishes (Table 9). There is no evidence of cointegration. The results of these experiments show that there is evidence of cointegration for the two series prior to the structural break but in the current period the two are not cointegrated. With respect to our research question, Super Restore and Prayer Potions are indeed substitutes for historical data but are not substitutes for recent data.

4.4 Breakpoints Model

In this brief section, we provide justification for using "2020-09-28" as a cutoff in Experiment 1. We use the `breakpoints()` function with the following specification:

$$S_t = \beta_0 + \beta_1 S_{t-1} + \beta_2 R_t + \beta_3 \text{Snap}_t + \beta_4 P_t + \epsilon_t.$$

For this model, we include all regressors for which we believe impacts the price of Super Restore. The covariates above were chosen because in the fixed effects model we found these variables to be influential to the price of Super Restore. We do not include Snapdragon Seed because in the VAR section, the impulse response suggested the two have a weak relationship. We include a lag for Super Restore because in the first section we saw evidence of serial correlation. Other variables are contemporaneous because the contemporaneous variables were used to estimate the edges in the value chain DAG.

The results of `breakpoints()` function suggest that only one breakpoint is necessary, as it yields the lowest BIC (Figure 20). The breakpoint selected was "2020-09-28", which aligns exactly with the cutoff in Experiment 1. The regression using these breakpoints is shown in Table 10. Most of the coefficients in this regression aren't of interest except for the coefficients on prayer potion. The coefficients and significance suggests that prior to "2020-09-28" Super Restore and Prayer Potions were positively correlated (substitutes) but after the "2020-09-28" they became inversely correlated (complements). The R^2 of the model is surprisingly high which suggests that the model has captured most of the variation in Super Restore but again we must be wary of any serial correlation as this inflates the R^2 . With this result, we conclude our analyses for research question (2); Prayer Potion and Super Restore are substitutes for older data but are not for newer data.

5 Forecast Modeling

5.1 Prophet

Most of the research questions are answered up to this point. For the remainder of the project we focus on finding the model that best forecasts Super Restore prices. We start with Prophet because the model makes it easy to implement holiday effects and structural breaks. Seeing how important they were in the prior section, it's vital we include them. We incorporate the structural break by including a dummy variable for dates past "2020-09-28". Our goal is to use Prophet with U.S. holiday effects and a structural break to forecast Super Restore. The reasoning for including holiday effects is because Runescape has many holiday events and tends to follow holidays in the U.S., since a large portion of the player base is in the U.S..

The decomposition of the Prophet model shows a negative trend and a large dip in price on Dec. 26 which is Boxing Day in the U.K. and the day after Christmas for the U.S.. There was a slight uptick prior to these holidays, which could indicate players rushing to finish the Christmas event before they became busy with the holidays. The dummy variable for the structural break is negative, which makes sense because Super Restore prices dropped a large amount following this date. The other seasonality components don't carry a lot of intuition. It seems like players are more active on weekdays than weekends but this isn't conclusive (Figure 23).

Finally, we perform a forecast on our test set. The resulting RMSE for the Prophet model is 1321.909. This is worse than the baseline model but not as bad as the VAR model.

5.2 ARIMA

Our penultimate forecasting model is ARIMA. Given how well it has done in the homework assignments, it is a model that cannot be ignored. We use `auto.arima()` to select the best model. Under the hood, the function selects the best model based on AIC. The chosen model was ARIMA(2, 1, 1). This specification implies a first-order differencing, with 2 lagged terms and one moving average term. The first-order differencing aligns with prior results, specifically the ADF tests, and the VAR and VECM models, both of which used growth rates.

$$\Delta y_t = \Phi_1 y_{t-1} + \Phi_2 y_{t-2} + \theta_1 \epsilon_{t-1} + \epsilon_t$$

The ARIMA model performed very well with an RMSE of only 154.592. The diagnostics are also incredibly good. The ACF contains no signs of serial correlation (Figure 24). The Ljung-Box test also fails to reject the null; no evidence of serial correlation. The distribution of residuals is symmetric and normal. From the standard errors in Table 11, all AR terms and MA terms are statistically significant. This ARIMA model also suggests the true lag structure of Super Restore is a lag order of two. If more time was given, a revision of the baseline model to use lag order of two would be in line. As mentioned, the lag order in the baseline model was chosen under the assumption that prices take longer to adjust but it seems like we underestimated the speed of price adjustments in a virtual world.

5.3 LSTM Model

Our final candidate forecasting model is an LSTM neural network (NN). This model is a large departure from all prior models because it is non-linear and is a deep learning model. There are other time series NN available but most underperform LSTM (Sagheer & Kotb, 2018). LSTM is similar to RNN in that it is recursive, but it has one additional design component called the LSTM unit. Some literature uses the metaphor of a "conveyor belt", which preserves old information and applies it when the model believes the old information will be useful. For example, if we are modeling payment transactions for golf courses, the LSTM can capture the winter closure structural break, save it, and use it for next year's forecast. Other models aren't able to capture this long-term dependency.

Prior to training, data is normalized (min-max scaled) and is manipulated to create a 14 day lag or window. This lag structure allows us to perform a supervised learning problem. We use the prior 14 days to predict the next day and so forth (sequence to vector), sliding the window forward, training the model. The batch size was set to a value of 16 after some trial and error and we used 50 epochs in total (Figure 25). No hyperparameters were tuned, as this is more of a prototype than a model.

The architecture of the model is kept simple. We use two hidden layers: the first with 100 units and the second with 100 units as well. The final layer is a dense layer which creates the prediction. The loss is a mean-squared error since RMSE is our performance metric. We generate the forecast and the resulting test RMSE was 151.341, which is just slightly below the ARIMA model (Figure 26). One caveat is that the R implementation of LSTM struggles with reproducibility. Therefore subsequent runs of the notebook may lead to different results.

6 Results

The table below summarizes the test RMSE for each of the forecasting model. ARIMA and LSTM were close in performance but the LSTM model has slightly better performance. LSTM performs best because it is able to capture peculiarities of the series along with being flexible enough to handle structural breaks. However, under the principle of Occam’s razor, we recommend ARIMA(2, 1, 1) as the best forecasting model.

Model	Test RMSE
OLS	223.882
VAR	11079.969
ARIMA(2, 1, 1)	154.592
LSTM	151.341
Prophet	1321.909

Revisiting our research questions we had:

1. Which model best forecasts Super Restore potions prices?
2. Are Prayer Potions and Super Restore Potions complement or substitute goods?
3. Is there a wealth effect on the price of Super Restore?
4. Do items in the virtual Runescape universe follow economic intuition?

We have shown with a degree of certainty that Super Restore and Prayer Potions are substitute goods using VAR impulse response functions and VECM. It is especially true for the period prior to "2020-09-28" since the cointegration and substitution relationship disappears after this breakpoint. We do not observe a wealth effect in our fixed effects regressions, since the models do not agree on the sign of the coefficients for the FRED variables. Finally, in all our analyses the items in the virtual economy reflect real-world economic intuition just at a much faster scale due to instantaneous transactions.

7 Conclusion

We have successfully answered all the research questions through empirical analysis. The implications show that Runescape acts very much like a real-world economy just at a faster clip. The item prices in this game are characterized by structural breaks and short-term momentum, as reflected by the performance of the fixed effects and ARIMA model. Items in this world are also found to be heavily dependent on one another.

7.1 Future Work

The project explored the gamut of models to answer the research questions presented. For brevity, there were a few models that were omitted from this paper, namely structural time series models and GARCH models. Structural time series is an interesting approach because it allows the parameters of the model to vary with time unlike models like ARIMA, which assume parameters are fixed. GARCH would be more tangential since it does not answer any of our research questions directly, focusing more on predicting the variance or volatility of a series.

This paper’s scope was narrow, focusing only on Super Restore. There are thousands of other items in the Runescape world with fresh data generated everyday. Hopefully other fans of the game will expand on this research, bringing forth interesting insights to this beloved online game.

7.2 Acknowledgments

The author would like to thank Professor William Yu and the TA Michael Brunjes for all the help they provided and for working tirelessly to provide the highest quality education possible. This project would not be possible without them.

References

[1] DeJong, D. N., Nankervis, J. C., Savin, N. E., & Whiteman, C. H. (1992). Integration versus trend stationary in time series. *Econometrica*, 60(2), 423. <https://doi.org/10.2307/2951602>

[2] Federal Reserve Economic Data | FRED | St. Louis Fed. (n.d.). <https://fred.stlouisfed.org/>

[3] Gagliardone, L., & Gertler, M. (2023). Oil prices, monetary policy and inflation surges.

[4] Gong, S., Levine, R., Lin, C., & Xie, W. (2023). Debtors at play: gaming behavior and consumer credit risk. *Management Science*. <https://doi.org/10.1287/mnsc.2023.4931>

[5] King, R. G., & Watson, M. W. (1996). Money, prices, interest rates and the business cycle. *The Review of Economics and Statistics*, 78(1), 35. <https://doi.org/10.2307/2109846>

[6] Mohr, F. (n.d.). An Introduction to Vector Error Correction Models (VECMs) · r-econometrics. <https://www.r-econometrics.com/timeseries/vecintro/>

[7] Parnes, D. (2019). Exploring economic anomalies in the S&P500 index. *The Quarterly Review of Economics and Finance*, 76, 292–309. <https://doi.org/10.1016/j.qref.2019.09.012>

[8] Runescape Grand Exchange data. (2024, September 19). Kaggle. <https://www.kaggle.com/datasets/aparoski/runescape-grand-exchange-data>

[9] Sagheer, A., & Kotb, M. (2018). Time series forecasting of petroleum production using deep LSTM recurrent networks. *Neurocomputing*, 323, 203–213. <https://doi.org/10.1016/j.neucom.2018.09.082>

[10] Server Population & Player Count - MMO populations. (2001, March 29). MMO Populations. <https://mmo-population.com/r/runescape>

[11] Sims, C. A. (1980). Macroeconomics and reality. *Econometrica*, 48(1), 1. <https://doi.org/10.2307/1912017>

[12] Vector error correction model output. (n.d.). Cross Validated. <https://stats.stackexchange.com/questions/383184/vector-error-correction-model-output>

A. Figures and Tables

A.1 Tables

Table 1: Raw Data Schema

Variable	Type	Description
index	<i>sequential int</i>	The primary key of the data set.
id	<i>int</i>	The id of an item.
price	<i>int</i>	The price of an item in gold (game currency).
volume	<i>int</i>	The number of trades for an item in a given day.
date	<i>timestamp</i>	The timestamp for which the record was created.

Table 2: Train/Test Data Schema

Variable	Type	Description
date	<i>timestamp</i>	Timestamp in yyyy-mm-dd format
year	<i>int</i>	Year as integer, i.e. 2012
month	<i>int</i>	Month as integer, i.e. 11
day	<i>int</i>	Day of the month as integer
prayer_potion	<i>double</i>	Daily series of prayer potion prices
snapdragon_seed	<i>double</i>	Daily series of snapdragon seed prices
snapdragon	<i>double</i>	Daily series of snapdragon prices
red_spiders_eggs	<i>double</i>	Daily series of red spider’s eggs prices
super_restore	<i>double</i>	Daily series of super restore potion prices
sp500	<i>double</i>	S&P 500 index (daily)
dgs10	<i>double</i>	10-year treasury yield (daily)

Table 3: Augmented Dickey–Fuller Test Results

Variable	Test Statistic	Test Result
Super Restore	-3.528**	Stationary
Red Spiders’ Eggs	-2.476	Non-stationary
Snapdragon	-2.646	Non-stationary
Snapdragon Seed	4.487	Non-stationary
Prayer Potion	-1.711	Non-stationary
S&P 500	-2.713	Non-stationary
10-Year Treasury Yield	-0.951	Non-stationary
	Significance	*0.05
		**0.01

Table 4: KPSS Test Results

Variable	Test Statistic	Test Result
Super Restore	4.527**	Non-stationary
Red Spiders’ Eggs	2.400**	Non-stationary
Snapdragon	3.558**	Non-stationary
Snapdragon Seed	6.176**	Non-stationary
Prayer Potion	16.491**	Non-stationary
S&P 500	2.435**	Non-stationary
10-Year Treasury Yield	2.886**	Non-stationary
	Significance	* 0.05
		**0.01

Table 5: Fixed Effect Model with Lags

	<i>Dependent variable:</i>
	super_restore_4
super_restore_lag_14	0.596*** (0.019)
super_restore_lag_28	0.132*** (0.019)
snapdragon	0.460*** (0.039)
snapdragon_lag_14	−0.184*** (0.049)
snapdragon_lag_28	−0.134*** (0.040)
sp500	−0.098* (0.054)
dgs10	45.435 (29.626)
Observations	2,860
R ²	0.988
Adjusted R ²	0.988
Residual Std. Error	474.385 (df = 2834)
F Statistic	9,290.271*** (df = 26; 2834)
<i>Note:</i>	*p<0.1; **p<0.05; ***p<0.01
† <i>Fixed effects were omitted for brevity</i>	

Table 6: DAG Estimation Model 1

	<i>Dependent variable:</i>
	super_restore_4
super_restore_lag_14	0.457*** (0.019)
super_restore_lag_28	0.161*** (0.019)
red_spiders_eggs	0.624*** (0.036)
red_spiders_eggs_lag_14	−0.087* (0.045)
red_spiders_eggs_lag_28	−0.018 (0.040)
sp500	0.164*** (0.051)
dgs10	13.516 (26.940)
Observations	2,860
R ²	0.990
Adjusted R ²	0.990
Residual Std. Error	440.808 (df = 2834)
F Statistic	10,776.740*** (df = 26; 2834)
<i>Note:</i>	*p<0.1; **p<0.05; ***p<0.01
† <i>Fixed effects were omitted for brevity</i>	

Table 7: DAG Estimation Model 2

	<i>Dependent variable:</i>
	clean_snapdragon
clean_snapdragon_lag_14	0.769*** (0.019)
clean_snapdragon_lag_28	0.018 (0.019)
snapdragon_seed	0.182*** (0.029)
snapdragon_seed_lag_14	−0.297*** (0.057)
snapdragon_seed_lag_28	0.186*** (0.039)
sp500	−0.025 (0.027)
dgs10	−46.450*** (14.041)
Observations	2,860
R ²	0.993
Adjusted R ²	0.993
Residual Std. Error	228.040 (df = 2834)
F Statistic	16,190.780*** (df = 26; 2834)
<i>Note:</i>	*p<0.1; **p<0.05; ***p<0.01
† <i>Fixed effects were omitted for brevity</i>	

Table 8: VECM - Truncated

	super_restore_4	prayer_potion_4
ECT	-0.0043(0.0021)*	0.0023(0.0015)
Intercept	-0.3886(1.8949)	0.1133(1.3630)

† *Lags were omitted for brevity*

Table 9: VECM - Full Data

	super_restore_4	prayer_potion_4
ECT	-0.0010 (0.0008)	0.0006 (0.0008)
Intercept	0.5413 (1.7552)	0.6074 (1.6637)

† *Lags were omitted for brevity*

Table 10: Segmented OLS

	<i>Dependent variable:</i>
	super_restore
break_datessegment1	−12.402 (8.723)
break_datessegment2	214.184*** (68.728)
break_datessegment1:lag_super_restore	0.986*** (0.003)
break_datessegment2:lag_super_restore	0.982*** (0.013)
break_datessegment1:red_spider	0.020*** (0.007)
break_datessegment2:red_spider	−0.085*** (0.023)
break_datessegment1:clean_snapdragon	0.009*** (0.003)
break_datessegment2:clean_snapdragon	0.013 (0.019)
break_datessegment1:prayer_potion	0.004* (0.002)
break_datessegment2:prayer_potion	−0.017*** (0.006)
Observations	2,887
R ²	1.000
Adjusted R ²	0.999
Residual Std. Error	98.188 (df = 2877)
F Statistic	576,051.200*** (df = 10; 2877)

Note: *p<0.1; **p<0.05; ***p<0.01

Table 11: ARIMA(2,1,1) Model Summary for Super Restore

Coefficient	Estimate	Standard Error (s.e.)
AR(1) (ar1)	1.4378	0.0247
AR(2) (ar2)	-0.4810	0.0171
MA(1) (ma1)	-0.9498	0.0212
Model Fit Metrics		
σ^2		7573
Log-Likelihood		-16989.06
AIC		33986.13
AICc		33986.14
BIC		34010

A.2 Figures

Figure 1: Google Interest Index - Runescape

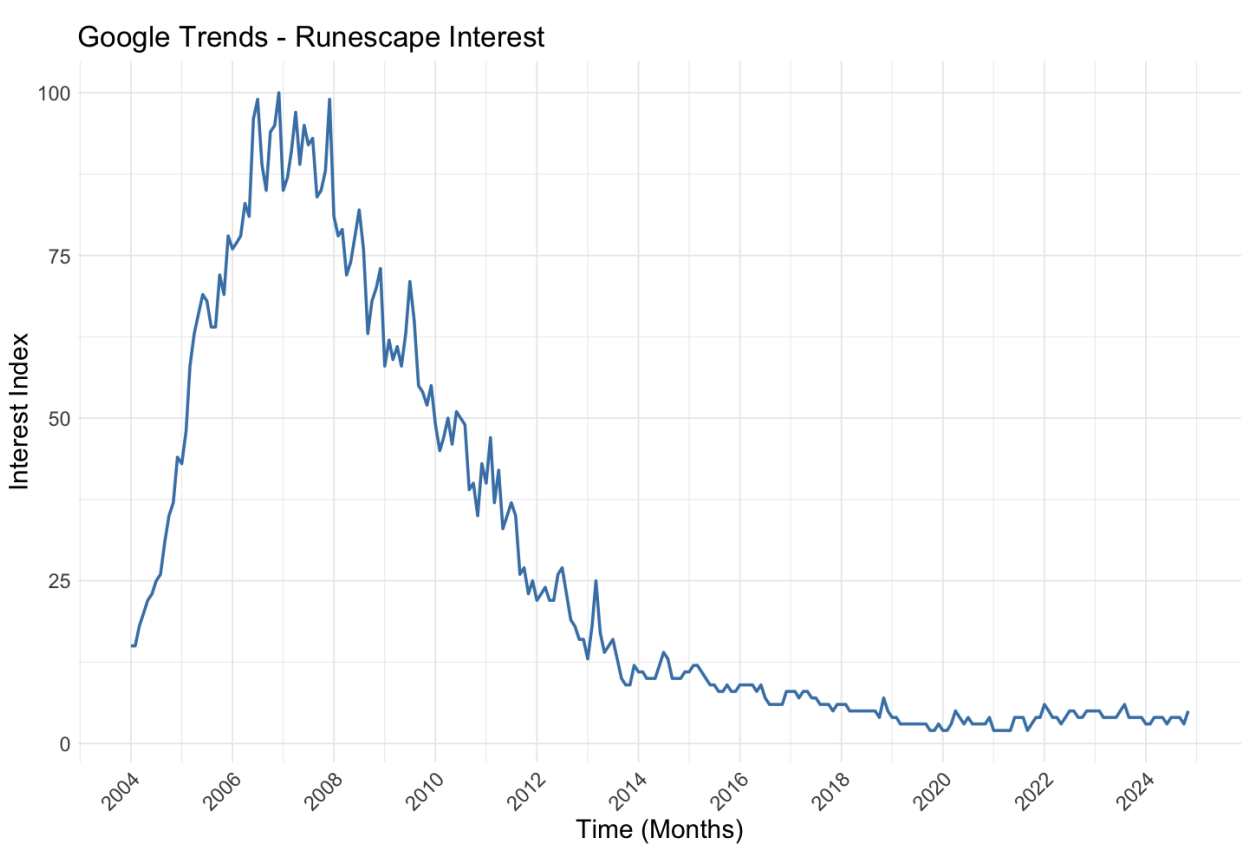


Figure 2: Missingness Percentage

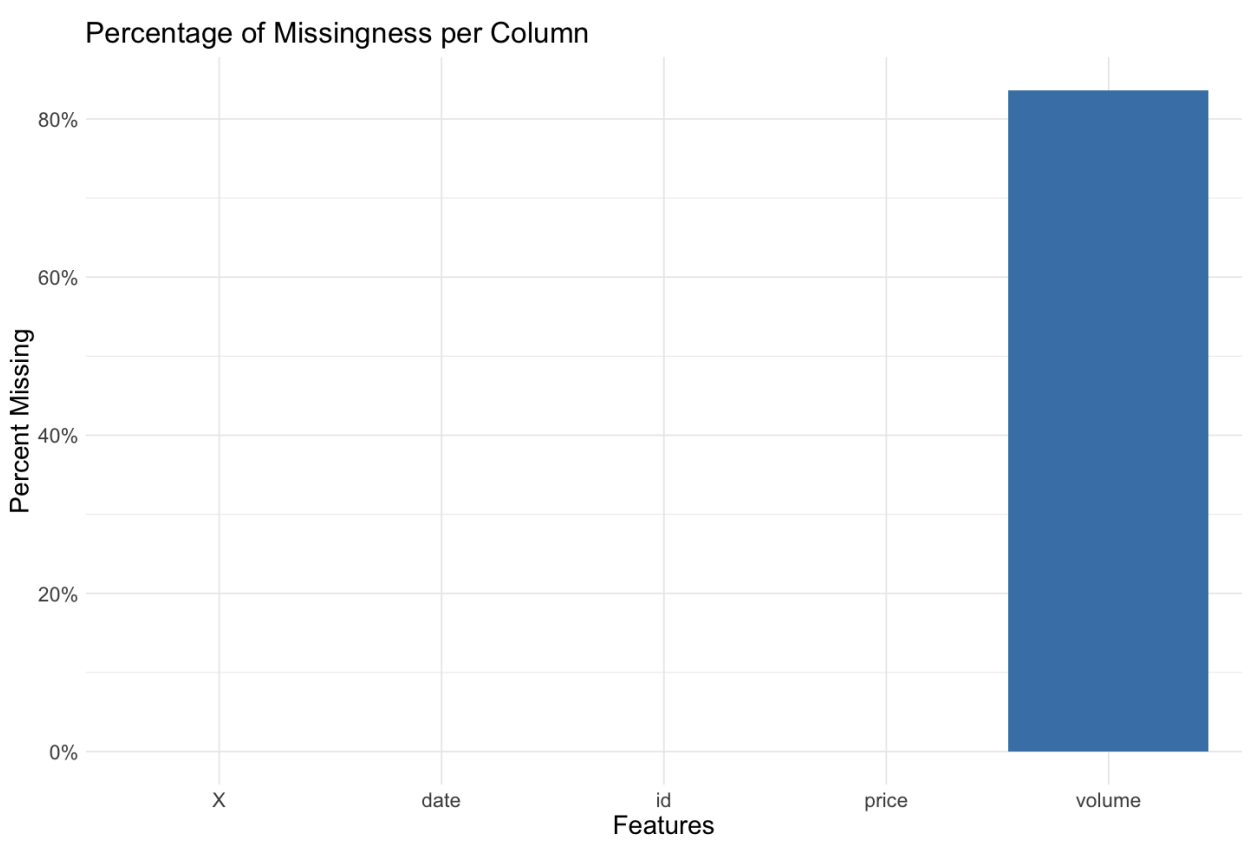


Figure 3: Count of Unique Items

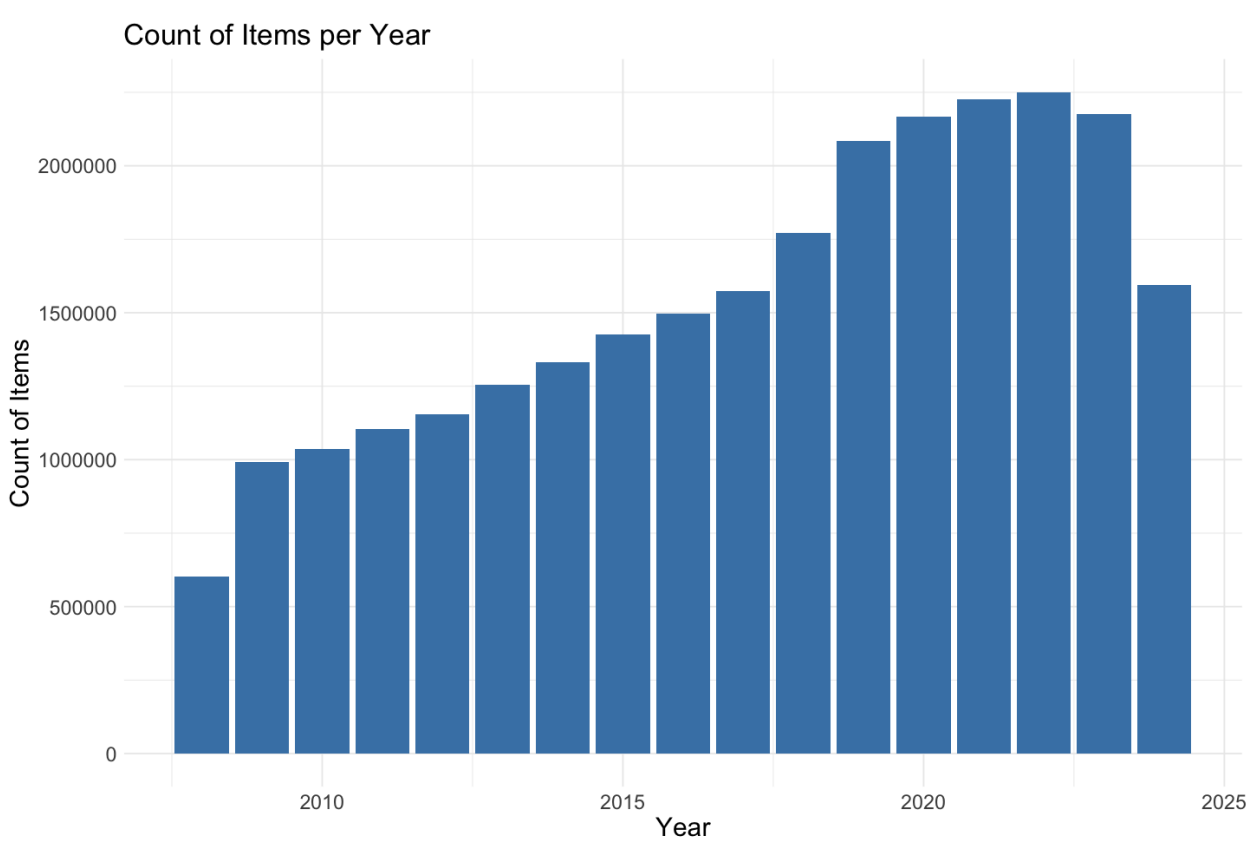


Figure 4: Aggregate Mean Price

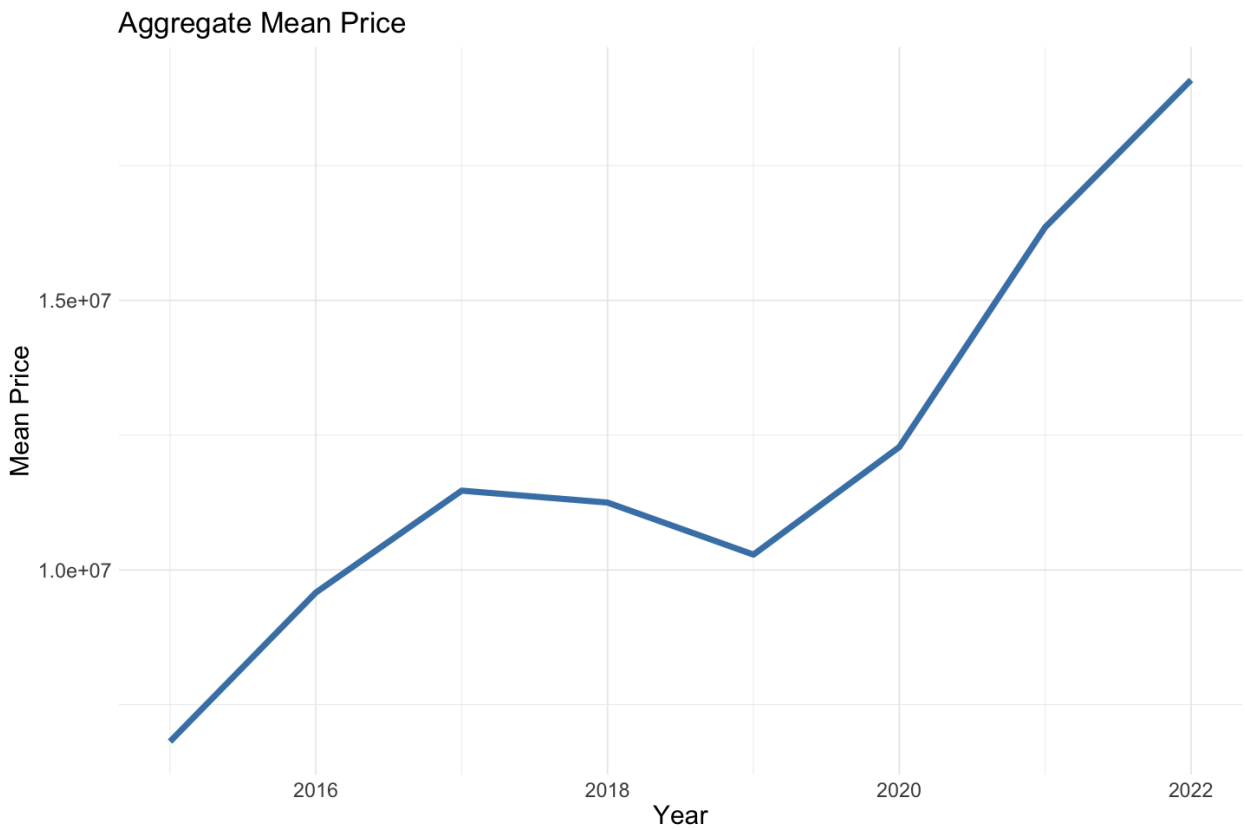


Figure 5: Value Chain DAG

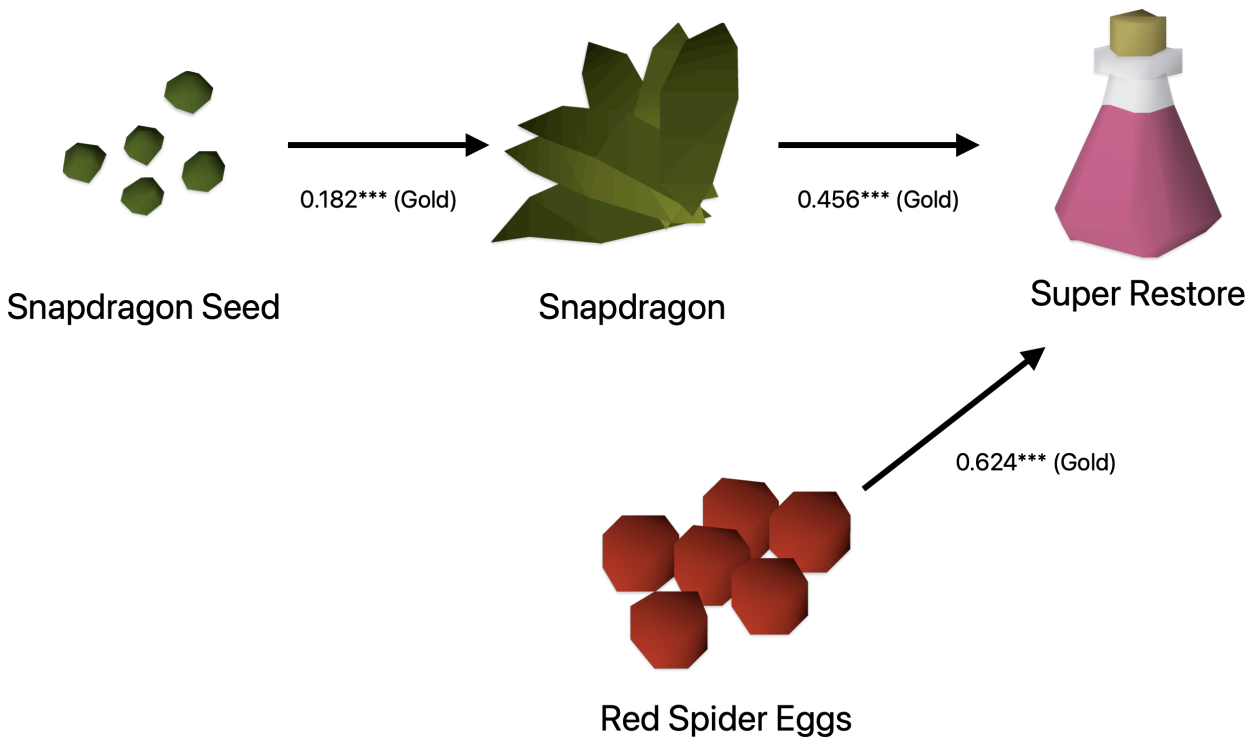


Figure 6: Prayer Potion Prices



Figure 7: Super Restore Prices

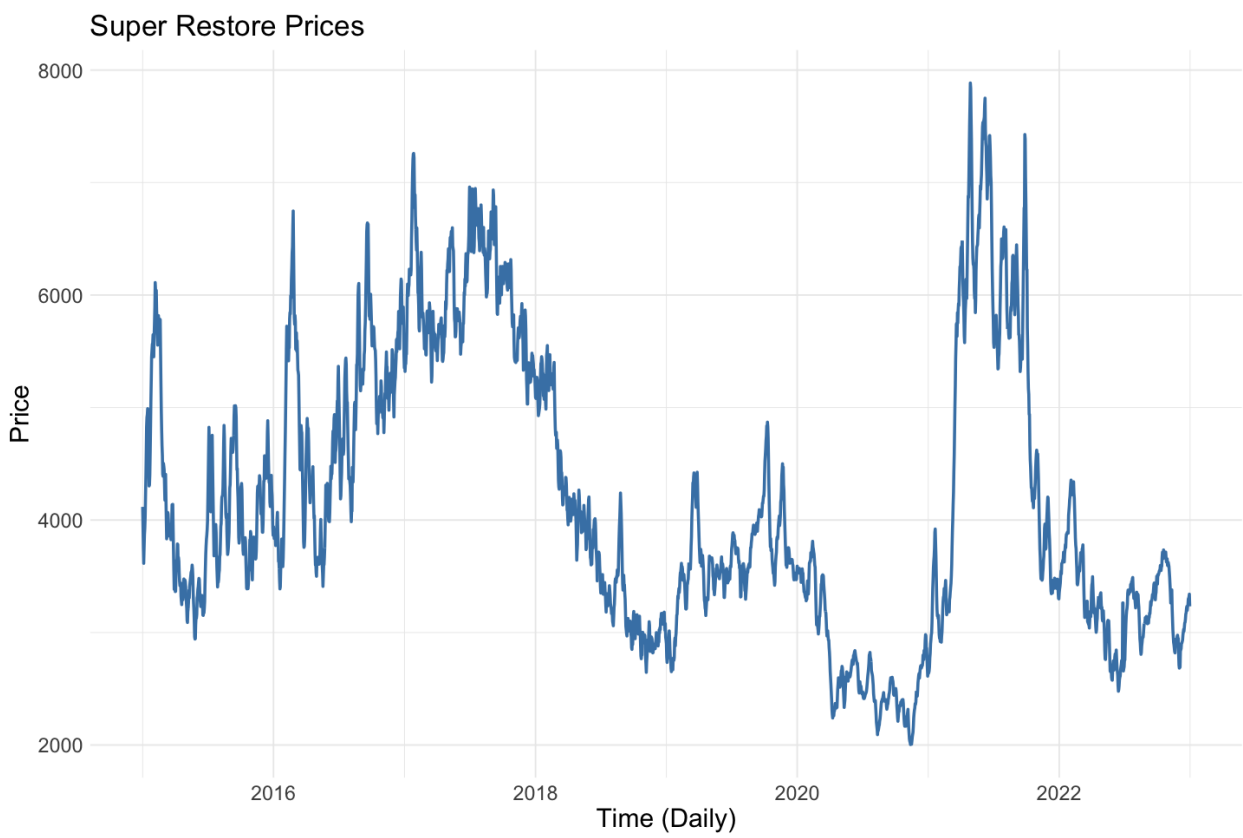


Figure 8: Snapdragon Prices

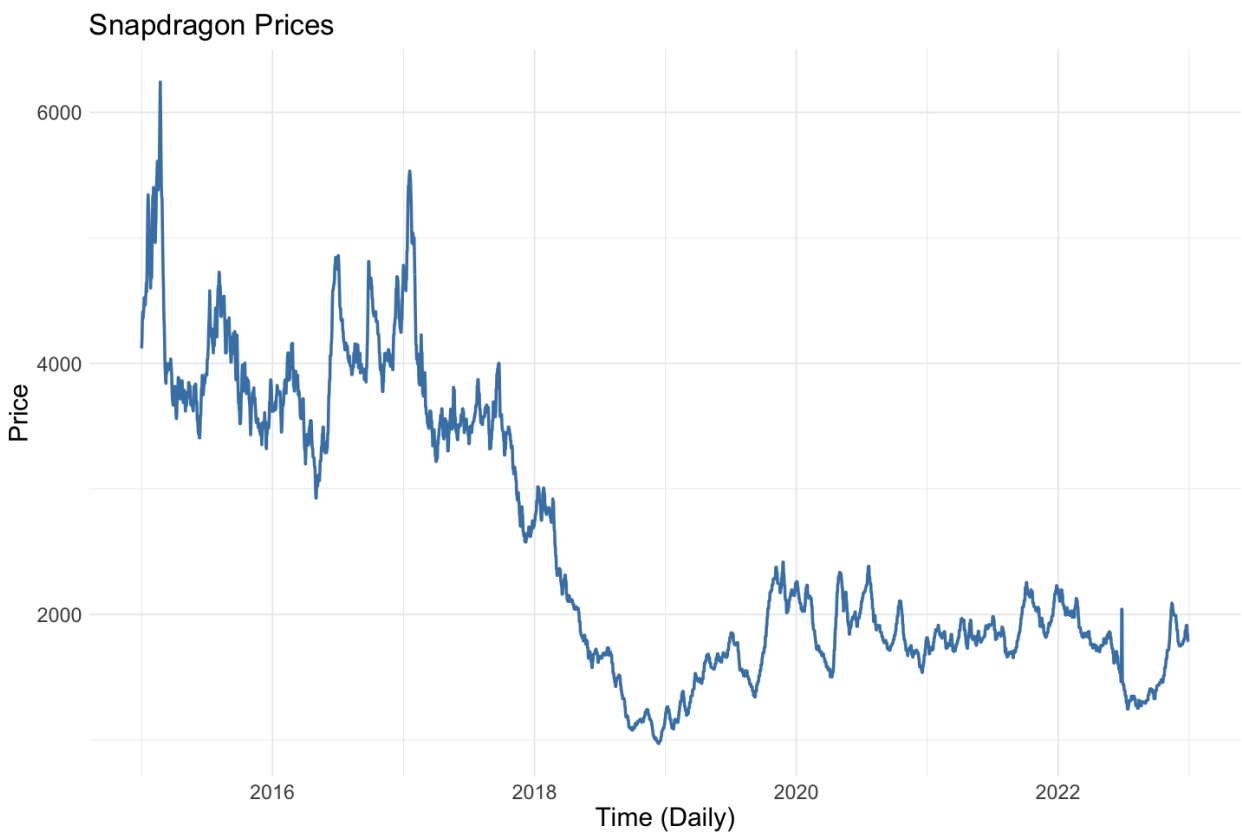


Figure 9: Snapdragon Seed Prices



Figure 10: Red Spiders' Eggs Prices

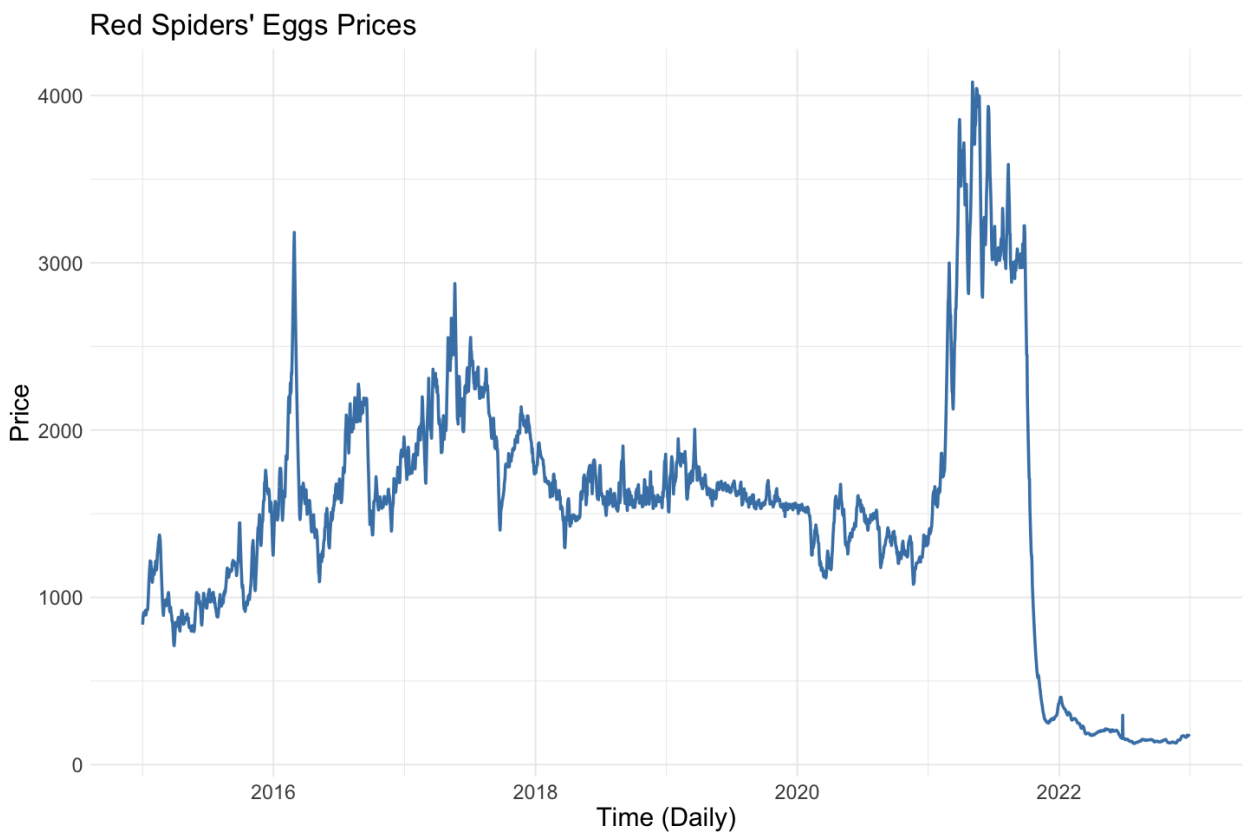


Figure 11: S&P 500 Index



Figure 12: 10-Year Treasury Yield



Figure 13: Fixed Effect Model Diagnostics

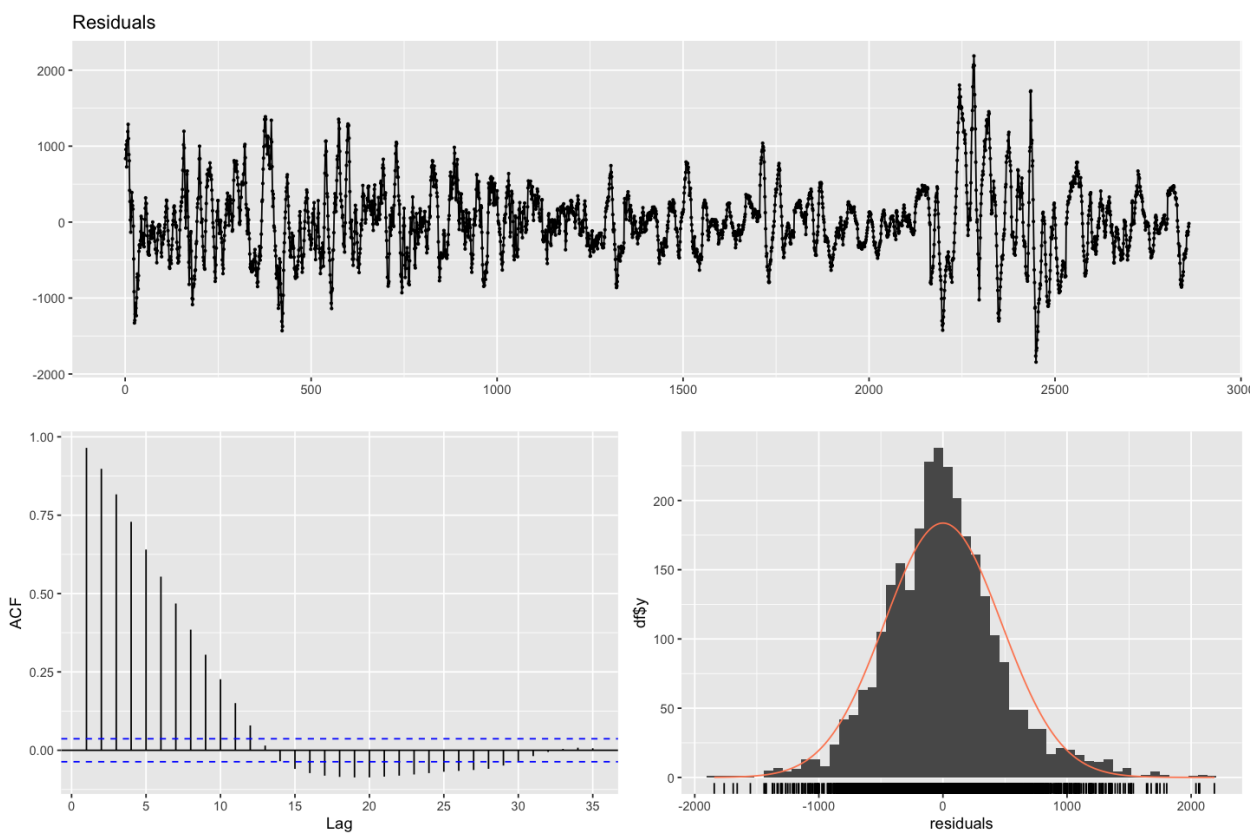


Figure 14



Figure 15

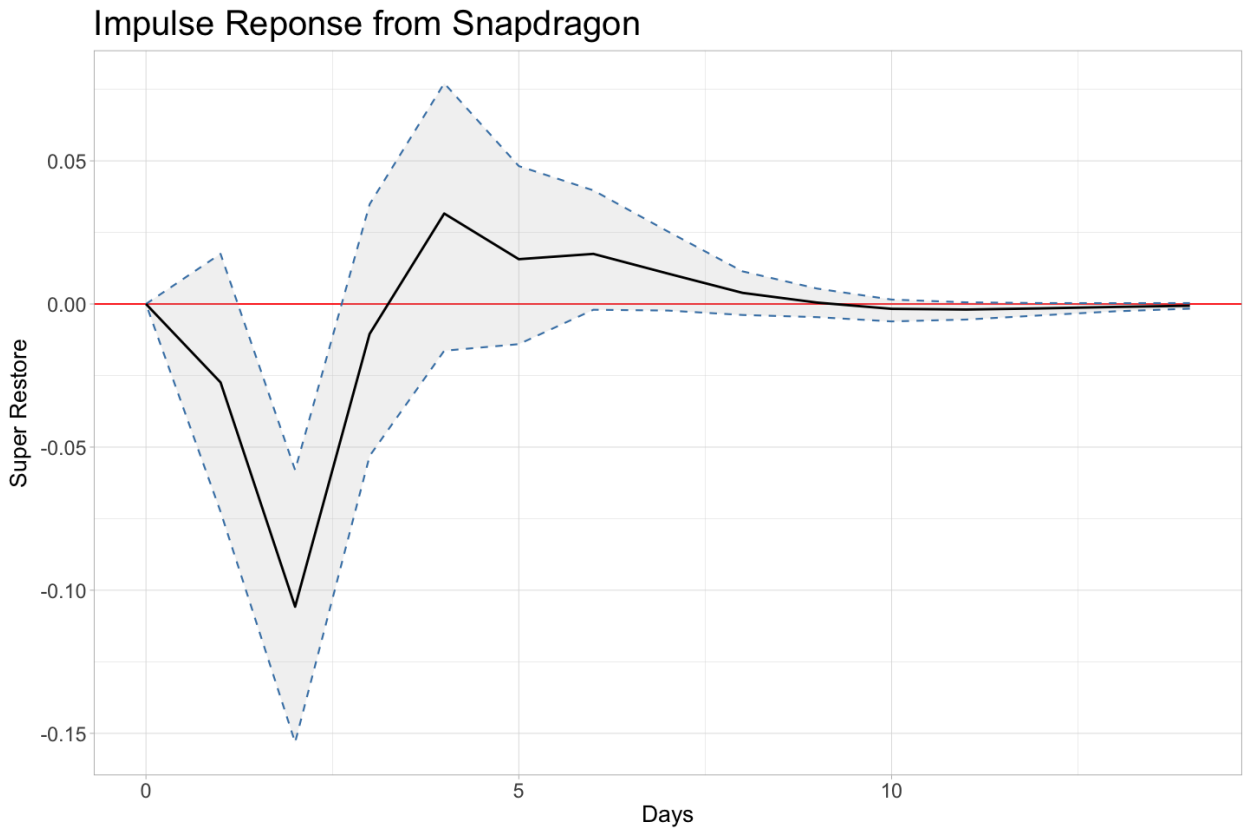


Figure 16

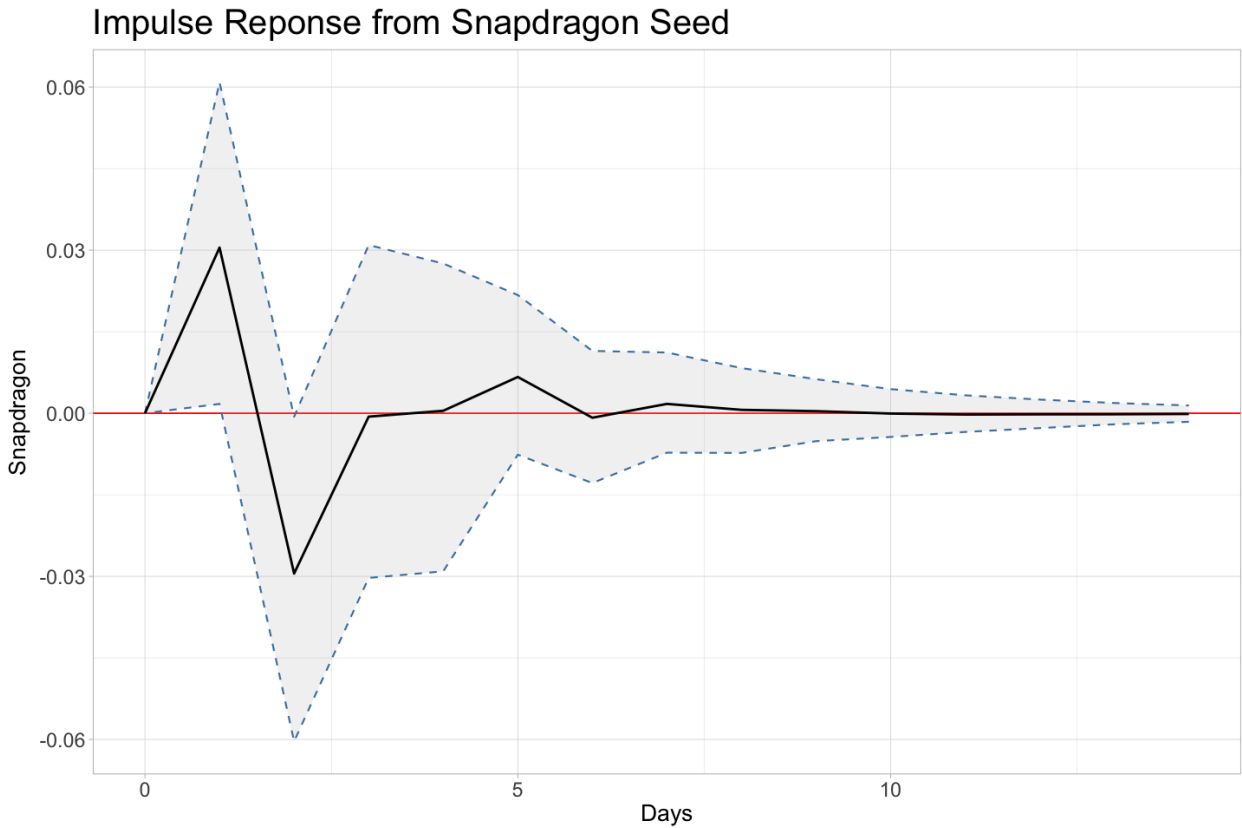


Figure 17

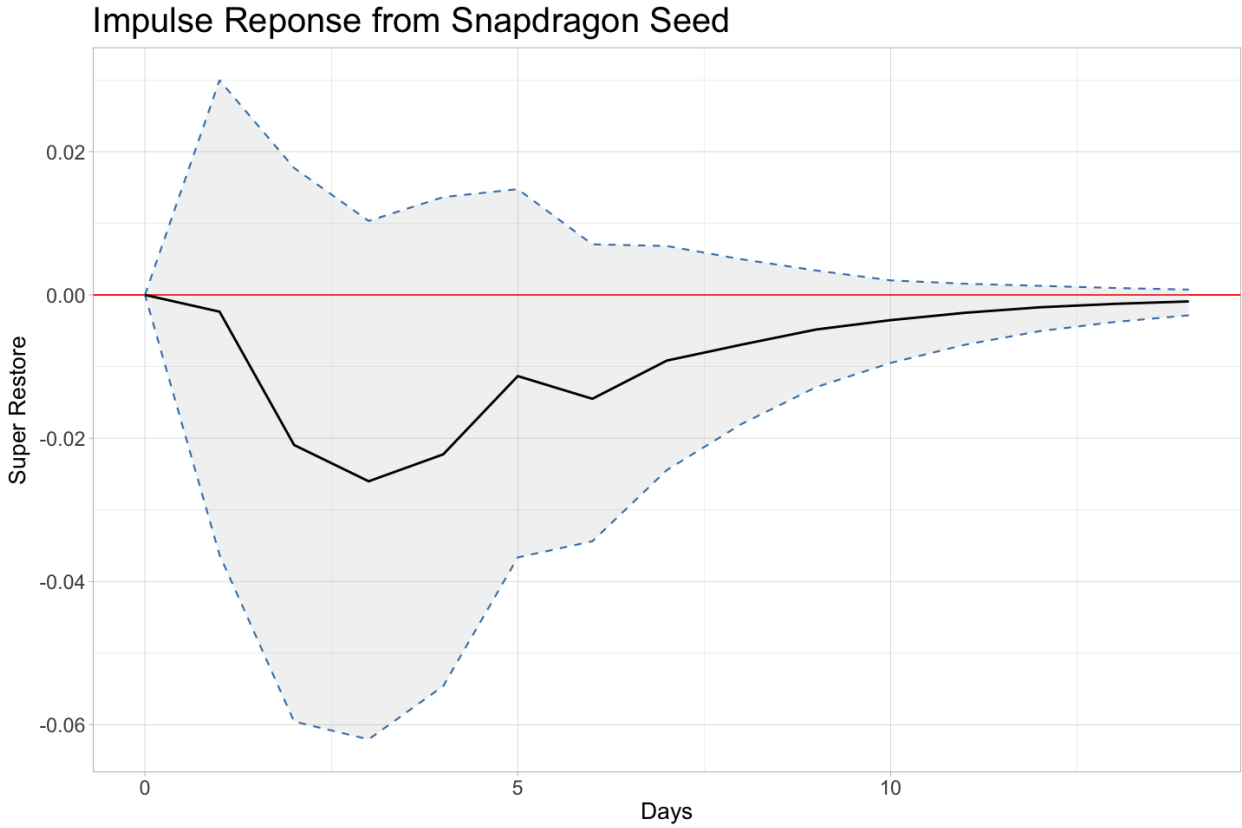


Figure 18

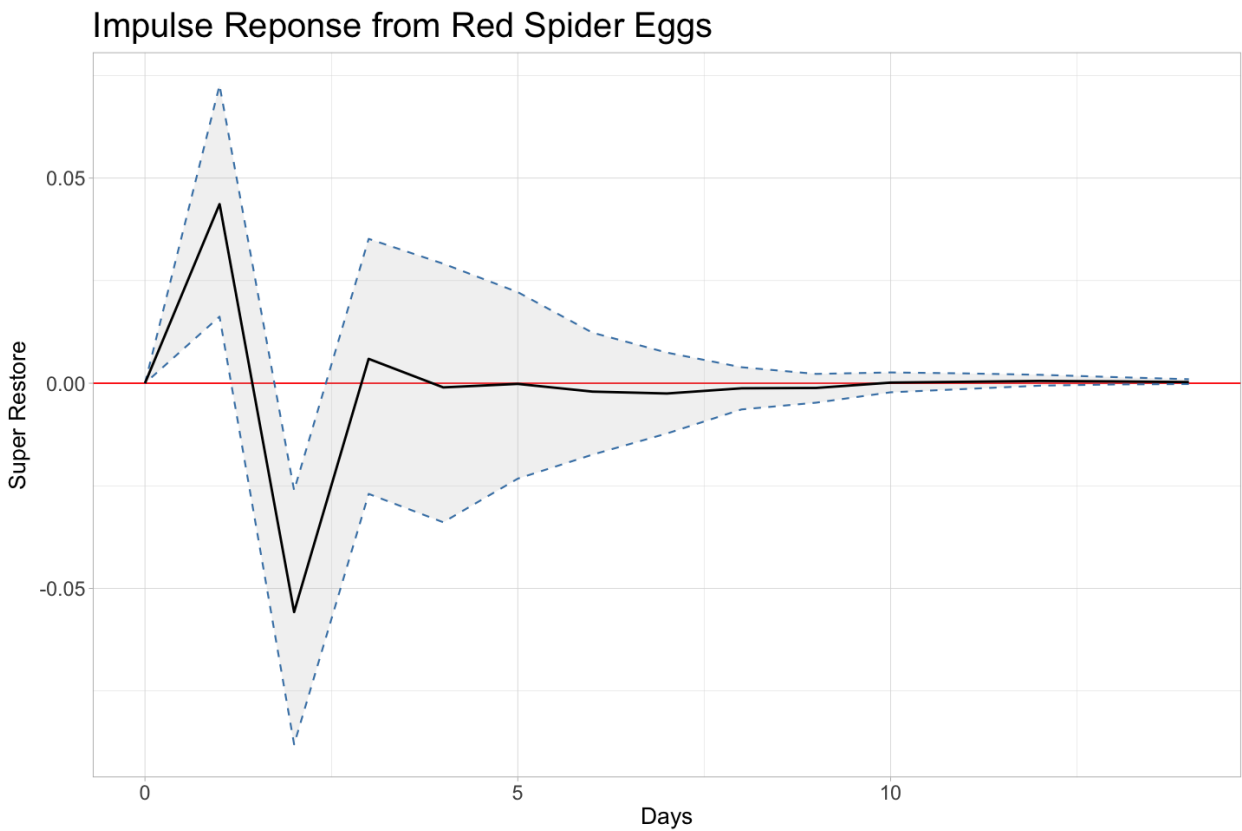


Figure 19

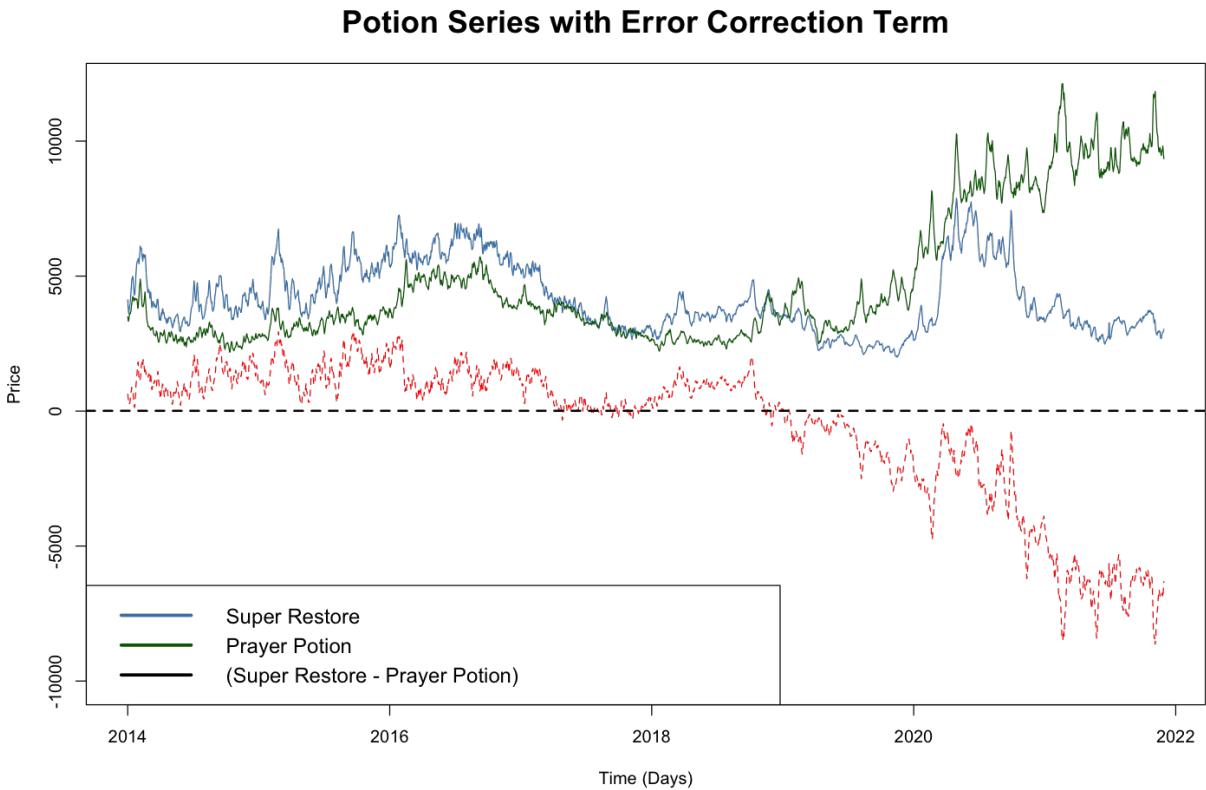


Figure 20: Optimal Breakpoints

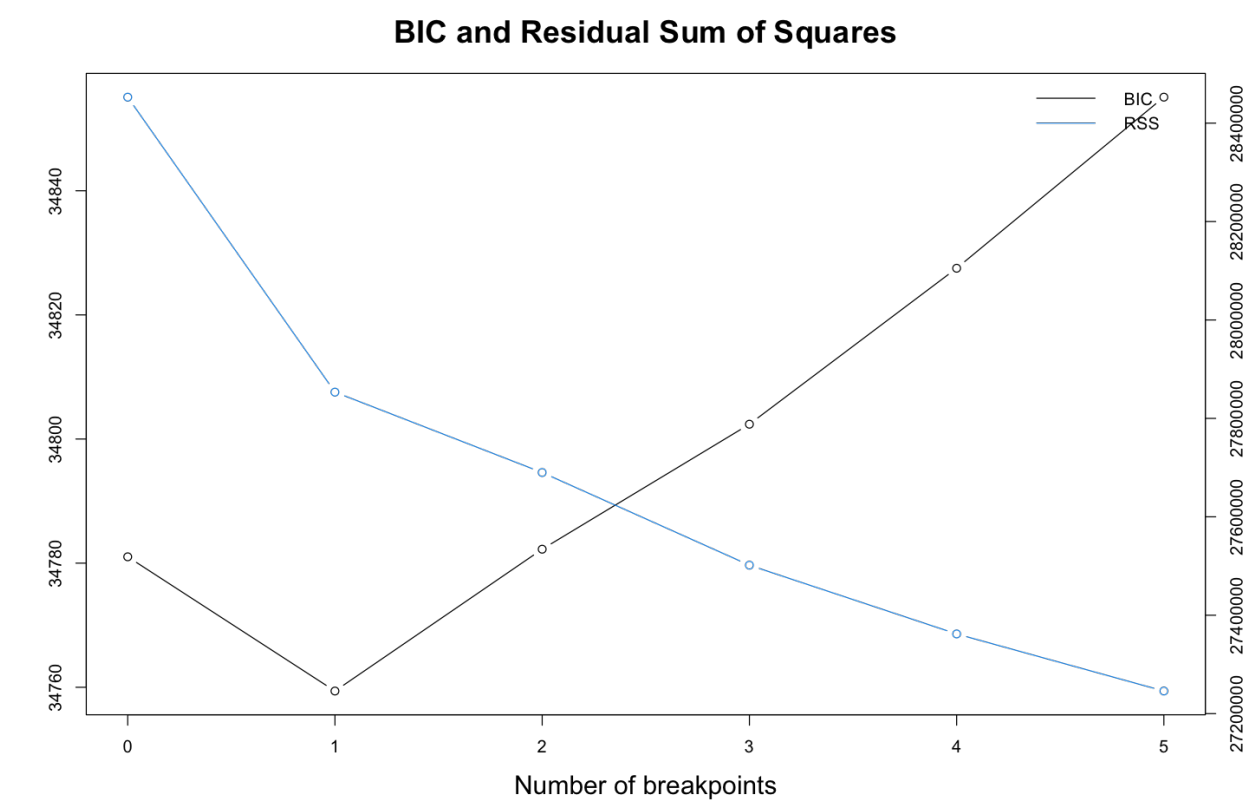


Figure 21: Super Restore Structural Break

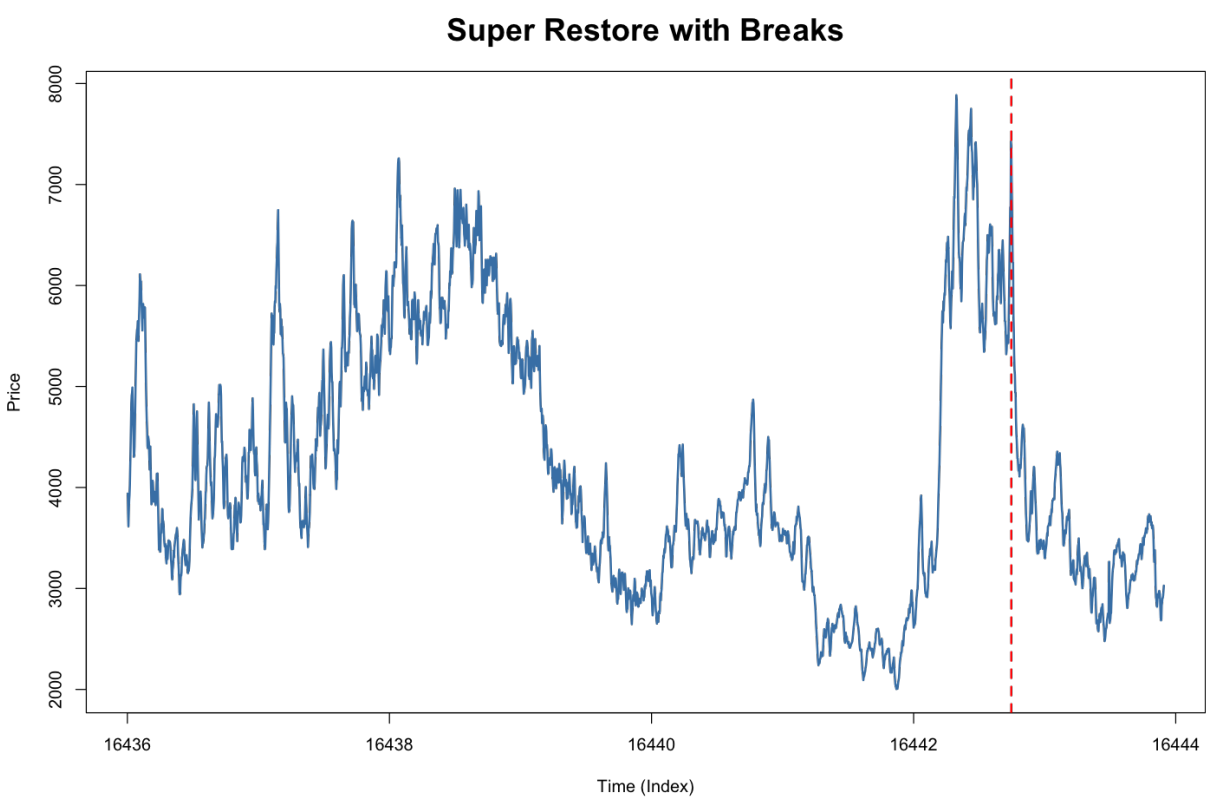


Figure 22: Prophet Model

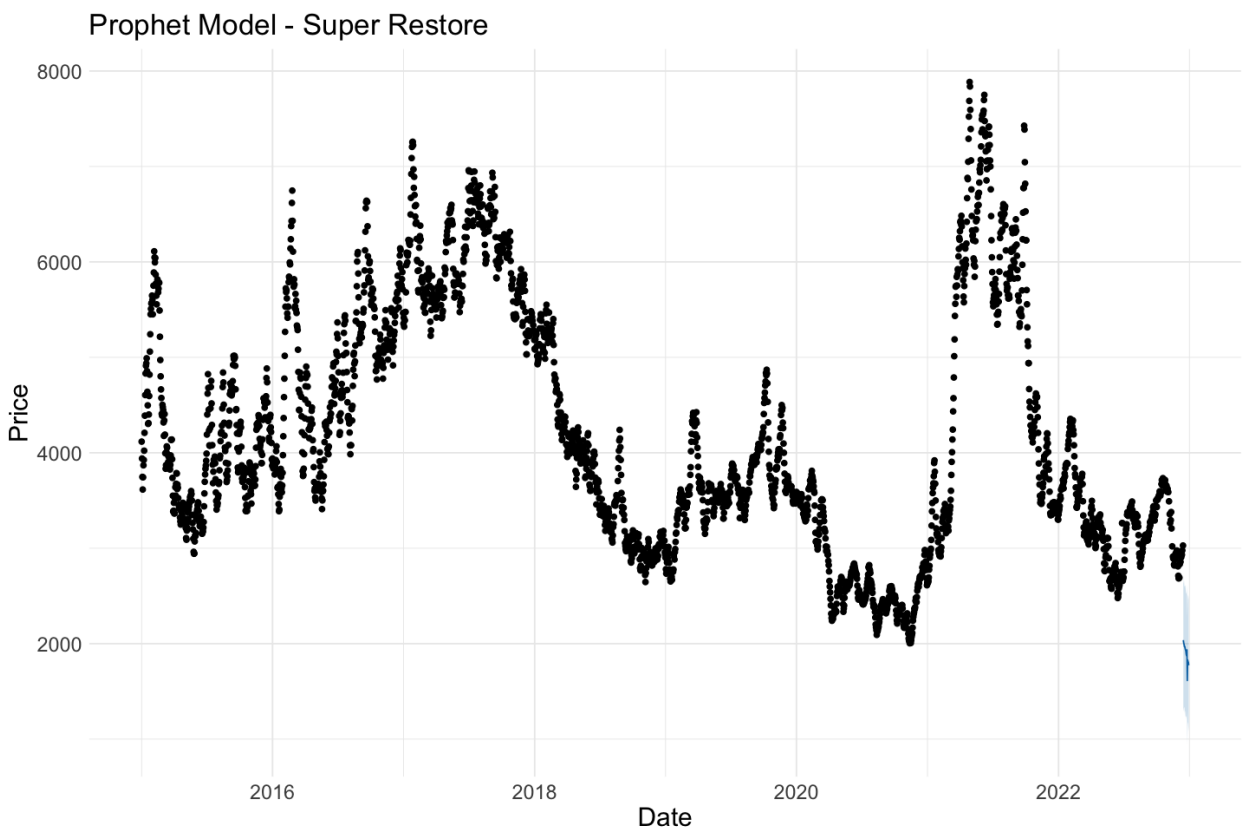


Figure 23: Prophet Model Decomposition

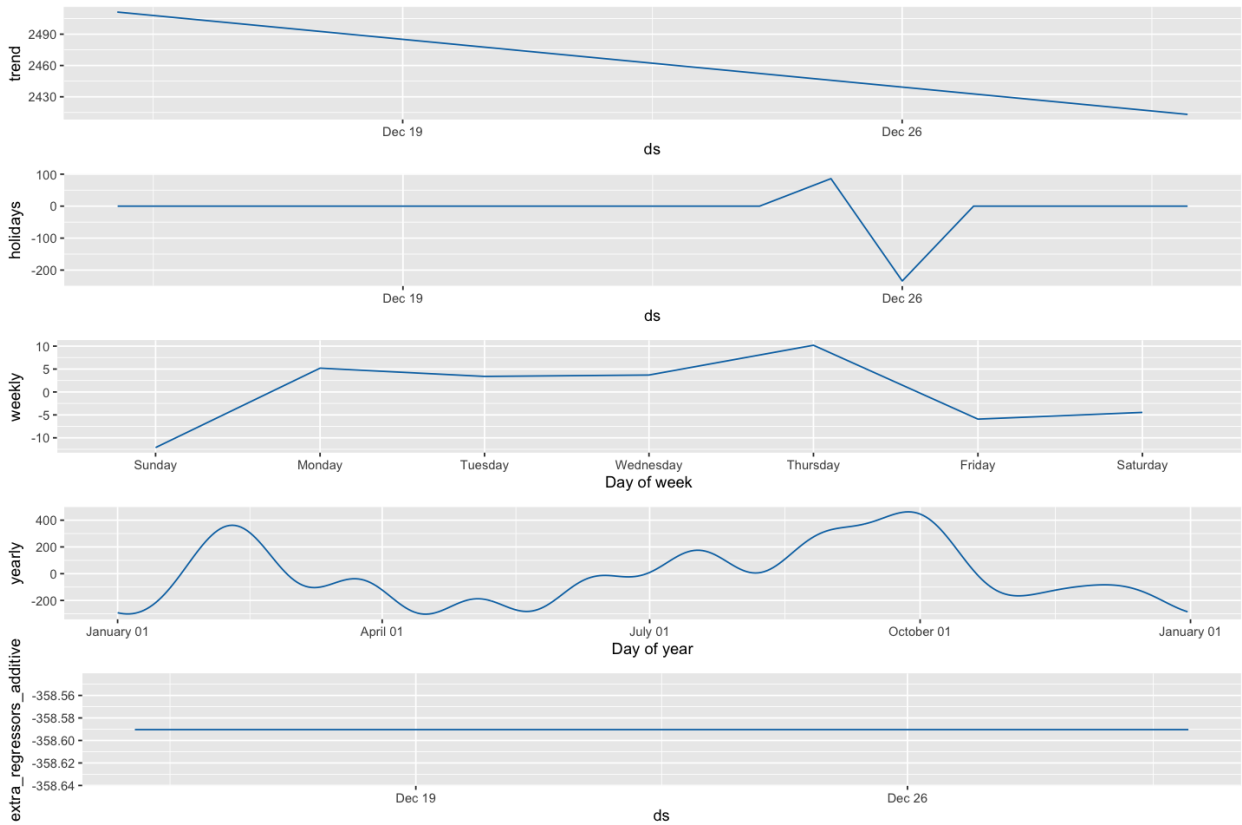


Figure 24: ARIMA(2, 1, 1) Diagnostics

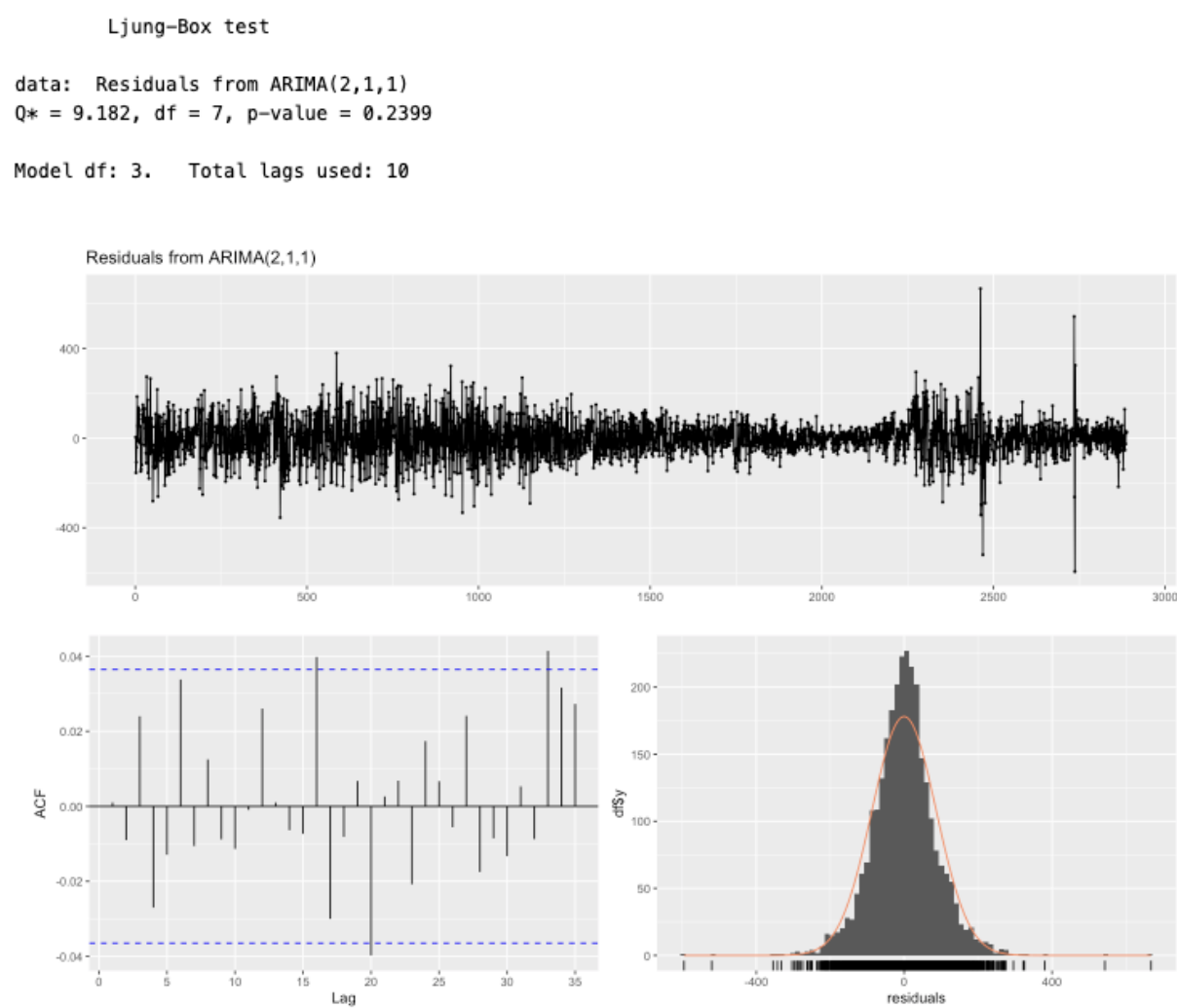


Figure 25: LSTM Training History

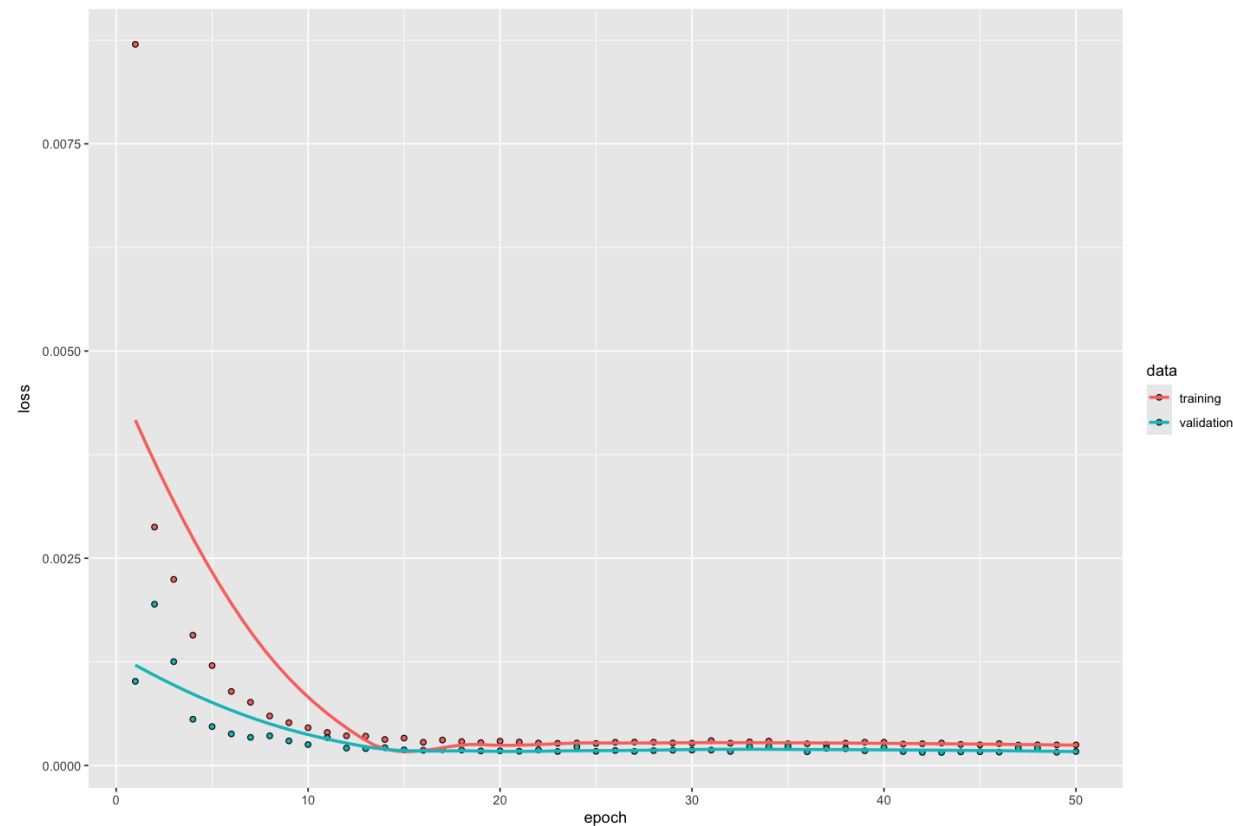
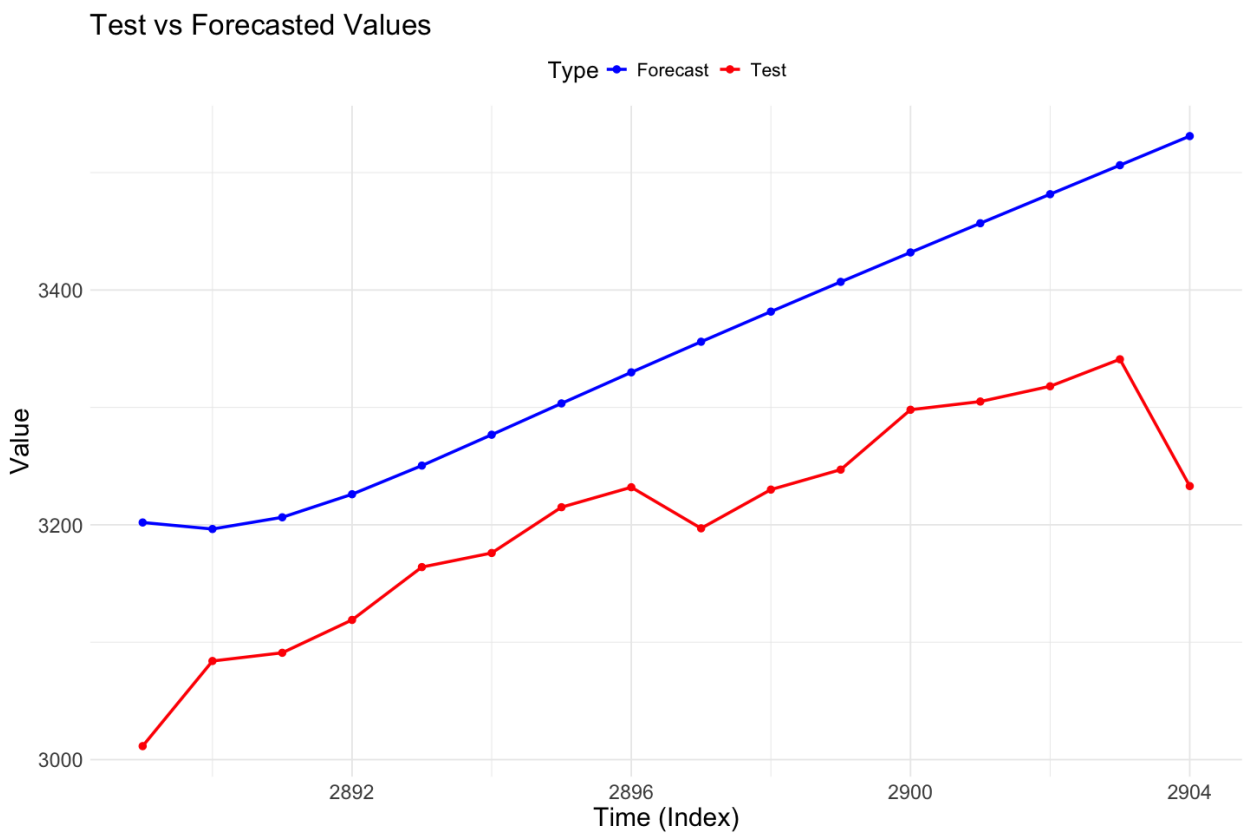


Figure 26: LSTM Forecast



B. Math Notation

Definitions

1. Stationarity

A random process $\{x(t), t \in \mathbf{R}\}$ is stationary if $\forall t_i \in \mathbf{R}$ and $\forall \Delta \in \mathbf{R}$ the joint CDF of

$$x(t_1), x(t_2), \dots, x(t_i)$$

is equal to the joint CDF of

$$x(t_1 + \Delta), x(t_2 + \Delta), \dots, x(t_i + \Delta).$$

In other words, $\forall x_i \in \mathbf{R}$

$$F_{x(t_1), x(t_2), \dots, x(t_i)}(x_1, x_2, \dots, x_i) = F_{x(t_1 + \Delta), x(t_2 + \Delta), \dots, x(t_i + \Delta)}(x_1, x_2, \dots, x_i)$$

, where F is the CDF.

2. Fixed Effects

For a model

$$y_{it} = X_{it}\beta + \gamma_i + \epsilon_{it}$$

The fixed effect γ_i is assumed to be independent of X_{it} for all time periods. This is the definition of strict exogeneity.

$$E[\epsilon_{it}|X_{it}] = 0$$

If this assumption holds, the fixed effect is able to control for unobserved heterogeneity.

3. Error Correction Model (ECM)

$$\Delta y_t = \Pi y_{t-1} + \sum_{i=1}^{p-1} \Gamma_i \Delta y_{t-i} + u_t$$

where Π is the adjustment matrix. For our case we only have two series, so Π will be 2×2 . The adjustment matrix can be decomposed further as $\alpha\beta^T$, where α is a $k \times r$ matrix of adjustment coefficients (speed of adjustment to equilibrium) and β is also a $k \times r$ matrix of cointegration vectors (models the long-run equilibrium). Typically $r=1$, which implies that the two original series are non-stationary but their linear combination is stationary.

Γ_i is the matrix of lags, d_t represents any exogenous terms and finally u_t is the error term that is Normal with mean 0 and an arbitrary covariance of Σ (Vector Error Correction Model Output, n.d.).

4. AIC

$$AIC = 2k - 2\ln(\hat{L})$$

Here k is the number of estimated parameters in the model and $\hat{L}$ is the maximized value of the model's likelihood function.

5. BIC

$$BIC = k\ln(n) - 2\ln(\hat{L})$$

Here k is the number of estimated parameters in the model, $\hat{L}$ is the maximized value of the model's likelihood function, and n is the number of observations.

The additional $\ln(n)$ term makes BIC a harsher metric than AIC. In AIC, the factor was fixed at 2.